Adapter Design Pattern

1. Classification, Intent & Exam Triggers

- **Category:** Structural Design Pattern (bridges two incompatible interfaces so existing classes can work together without modifying their source code).
- **Intent (GoF Definition):** Converts the interface of an existing class (Adaptee) into another interface (Target) that the client expects. Adapter lets classes work together that could not otherwise communicate because of incompatible interfaces.
- **Core Design Mantra:** Don't change existing code — Adapt it! The Adapter acts as a middleman translator (wrapper class) that receives calls in the Client's expected format and translates them into the format required by the Adaptee.
- **Primary Problem Solved (Incompatible Legacy / Third-Party Interfaces):** Solves compilation and integration failures (e.g., Client calls 'printDocument()' or passes an 'Object', whereas the new/external service only provides 'printPDF()' or expects 'Json' / 'XML') when neither the Client nor the external/legacy class can be rewritten.
- **Core OOP Principles:** Applies Interfaces & Abstraction (Client programs strictly against the Target interface), Realization / Polymorphism (Adapter implements the Target interface), Composition ('has-a': Adapter holds a private reference to the Adaptee object and delegates translated calls to it), Encapsulation (hides data-format conversion like 'toJson()' inside the Adapter), and Open/Closed Principle (integrates new third-party vendors by adding new Adapter classes without modifying existing client code).
- **How to Spot in Exam Questions:** Look for trigger phrases such as 'incompatible interfaces,' 'different method signatures / parameter formats (e.g., XML vs. JSON, IR vs. Bluetooth, Wi-Fi vs. Zigbee, Object vs. JSON),' 'integrate an existing legacy class or external vendor / third-party web service that cannot be modified,' or 'switch to a new payment/shipping/printer provider without rewriting existing application code.' Typical question stem: 'An existing application expects a specific interface, but the organization wants to integrate a legacy module or external third-party service whose method names or data formats are different and cannot be modified. Which design pattern should be used?'

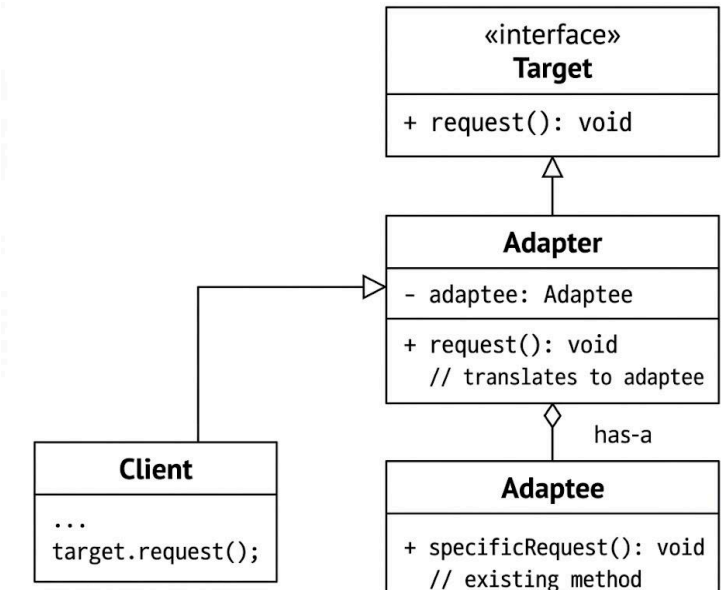
2. Pattern Structure, Participants & UML Relationships

Participant Role	Responsibility in Pattern
1. Target (Expected Interface)	Defines the domain-specific interface (method contract) that the Client already knows and uses.
2. Adaptee (Incompatible Class)	The existing legacy class, new hardware driver, or external third-party library/web service that has an incompatible interface (different method names or data types) and cannot be modified.
3. Adapter (Translator / Wrapper)	Implements the Target interface AND holds a reference ('has-a' composition) to the Adaptee. Translates client calls on the Target method into compatible calls on the Adaptee.

4. Client (Existing Application)	Collaborates with objects conforming to the Target interface. Remains completely unaware of the Adaptee and its internal interface.

UML Relationships & Essential Design Q&A Rules

- **Realization / Implementation ('is-a' — dashed line with hollow triangle pointing to Target):** The Adapter class implements the «interface» Target (e.g., WebAdapter implements WebRequester; PrinterAdapter implements Printer).
- **Composition / Association ('has-a' — solid line with diamond on Adapter pointing to Adaptee):** The Adapter wraps and holds a private field referencing the Adaptee (e.g., private WebService service; or private ModernPrinter modernPrinter;).
- **Client Dependency (solid arrow: Client -> Target Interface):** The Client references ONLY the Target interface (e.g., private WebRequester webRequester;) and never links directly to the Adaptee.
- **Q&A Rule 1 — If the interfaces are incompatible, why can't we just change the Adaptee (Web Service / Legacy Class)?** Because the Adaptee is usually supplied by an external third-party vendor (closed-source library, remote API, or hardware driver) where we do not own the source code, OR it is a battle-tested legacy system that other critical modules depend on and modifying it would risk regression bugs.
- **Q&A Rule 2 — Why can't we change the Client code to match the new Web Service / Adaptee directly?** Because the Client application is already built around the Target interface across many modules. Rewriting the Client tightly couples it to one vendor's proprietary API, violates the Open/Closed Principle, and forces a full application rewrite every time you switch vendors in the future.
- **Q&A Rule 3 — Who knows the Adaptee, and does Adapter change the Client?** Only the Adapter knows the Adaptee. The Adapter does NOT change the Client at all — the Client continues calling the Target interface as before.



3. Implementation Code Structure

```
// Step 1: Design the Target Interface expected by the Client
public interface WebRequester {
  public int request(Object requestData);
}

// Incompatible Adaptee (Third-Party WebService that only accepts and returns Json)
public class WebService {
  private String webHost;
  public WebService(String webHost) { this.webHost = webHost; }
  public Json request(Json data) {
    System.out.println("WebService processing JSON payload at " + webHost);
    return data; // Returns JSON response
  }
}

// Step 2: Implement the Target Interface with the Adapter Class (WebAdapter)
public class WebAdapter implements WebRequester {
  private WebService service; // Composition (has-a reference to Adaptee)

  public void connect(WebService currentService) {
    this.service = currentService;
  }

  @Override
  public int request(Object requestData) {
    // Translate Client's Object into Json expected by the WebService Adaptee
    Json result = this.toJson(requestData);
    Json response = service.request(result);
    if (response != null) {
      return 200; // HTTP OK status code
    }
    return 500; // Server error status code
  }

  private Json toJson(Object input) {
    // Converts arbitrary Java Object into Json representation
    return new Json(input.toString());
  }
}

// Step 3: Client Class (WebClient) — Interacts ONLY with the WebRequester Target Interface
public class WebClient {
  private WebRequester webRequester;

  public WebClient(WebRequester webRequester) {
    this.webRequester = webRequester;
  }
```

```
  private Object makeObject() { return new Object(); }

  public void doWork() {
    Object object = makeObject();
    int status = webRequester.request(object);
    if (status == 200) {
      System.out.println("OK");
    } else {
      System.out.println("Not OK");
    }
  }
}

// Step 4: Main Program Wiring
public class Program {
  public static void main(String[] args) {
    String webHost = "Host: https://google.com\n\r";
    WebService service = new WebService(webHost); // 1. Create Adaptee
    WebAdapter adapter = new WebAdapter();       // 2. Create Adapter
    adapter.connect(service);                    // 3. Connect Adapter to Adaptee
    WebClient client = new WebClient(adapter);   // 4. Pass Adapter as Target to Client
    client.doWork();
  }
}
```

Built-In Java Standard Library Adapters (Quick Reference)

- **java.io.InputStreamReader:** Adapts a byte-based InputStream (Adaptee, e.g., FileInputStream) into a character-based Reader (Target Interface): 'Reader reader = new InputStreamReader(new FileInputStream("data.txt"));'.
- **java.util.Arrays.asList():** Adapts a fixed-size primitive/object Array 'T[]' (Adaptee) into a 'List<T>' collection interface (Target): 'List<String> list = Arrays.asList(names);'.
- **java.util.Collections.enumeration():** Adapts a modern 'Collection<E>' (Adaptee) into a legacy 'Enumeration<E>' interface (Target).

4. Key Comparisons, Trade-Offs & Real-World Challenges

Adapter vs. Facade, Decorator & Proxy

Comparison Area	Adapter Design Pattern (Translator)	Counterpart Pattern
vs. Facade (Primary Purpose)	Makes two existing incompatible interfaces compatible so they can work together (Alters/Translates interface).	Facade: Simplifies interaction with a complex subsystem by providing a single unified front-door interface.
vs. Facade (Scope & Target)	Wraps ONE incompatible object (Adaptee) to match an	Facade: Coordinates MANY subsystem objects behind a

	already-expected Target interface.	brand-new simplified interface.
vs. Decorator (Interface & Intent)	Changes/converts the interface from Target to Adaptee without adding new business behavior.	Decorator: Keeps the EXACT same Component interface and dynamically adds new responsibilities/behaviors.
vs. Proxy (Interface & Intent)	Bridges two DIFFERENT interfaces (Target != Adaptee).	Proxy: Implements the EXACT same interface as the RealSubject to control access, security, or lazy loading.

Advantages, Disadvantages & Usage Rules

- **Key Advantages:**

  (1) Reuses Legacy & Third-Party Systems: Allows stable legacy classes and external vendor libraries with incompatible interfaces to be reused without modifying their source code.

  (2) Loose Coupling: Client code depends purely on the Target interface, remaining decoupled from the concrete Adaptee's implementation and data formats.

  (3) Open/Closed Principle (OCP): New vendor services or hardware drivers can be integrated by introducing a new Adapter class with zero changes to existing client code.

  (4) Easy Unit Testing: Because the Client uses the Target interface, the Adapter/Adaptee can be easily mocked or stubbed during unit testing.
- **When to Use:** When integrating third-party libraries, connecting to external web services with different method signatures or data formats (JSON/XML), upgrading hardware/drivers, or reusing legacy modules that cannot be changed.
- **When NOT to Use:** When you own and can easily refactor both classes directly during initial design, or when excessive adapters are chained together instead of standardizing internal interfaces.

Real-World Challenges & Mechanisms to Overcome Them

Challenge Aspect	How to Explain & Mechanism to Overcome in the Exam
1. Writing Additional Code & Class Proliferation **('Adapter needs more code — Do you agree?')**	Challenge: Yes, creating a separate Adapter class for every incompatible third-party vendor (e.g., 20 different hotel or airline web services) requires writing extra wrapper classes and increases package complexity. Mechanism to Overcome: The payoff is well worth it because it preserves the stability of unmodifiable legacy/vendor systems without touching client code. To reduce class bloat, combine Adapter with a Factory or configurable mapping registry.
2. Translation & Serialization Performance Overhead	Challenge: Every request passes through an extra indirection hop, and complex data-format conversions (such as parsing/serializing large Object graphs to JSON or XML) add runtime latency and CPU overhead. Mechanism to Overcome: Use lightweight streaming parsers, cache

	frequently translated responses, or perform bulk/batch data adaptation rather than translating item-by-item.
3. Two-Way / Bi-Directional Adaptation & Mismatched Capabilities	Challenge: Sometimes an Adaptee lacks a direct equivalent for a Target method (or throws different exceptions/status codes), making 1-to-1 translation incomplete. Mechanism to Overcome: Standardize error/status mapping inside the Adapter (e.g., mapping null/exceptions to HTTP 500 vs. 200) and provide fallback default values or Two-Way Adapters when both systems must call each other.
4. Managing Multiple Subsystems That Also Use Third-Party Vendors	Challenge: In large portals (like a Foreign Trip Booking Website combining Air Ticket, Hotel, and Taxi subsystems), the client faces both multi-subsystem complexity AND incompatible third-party vendor APIs. Mechanism to Overcome: Combine Facade + Adapter! Use a Facade at the top layer to coordinate the overall booking workflow, and use Adapters at the bottom layer of each subsystem to normalize each third-party vendor's web service signature.